P2642R1
Padded mdspan layouts

Published Proposal, 2022-10-15

This version:
https://github.com/ORNL/cpp-proposals-pub/blob/master/layout_padded/layout_padded.bs

Author:
Mark Hoemmen (NVIDIA)

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose two new mdspan layouts, layout_left_padded and layout_right_padded. These are strided layouts where the leftmost resp. rightmost extent is always stride 1, but the next stride to the right resp. left can be larger than the leftmost resp. rightmost extent. The new layouts can represent this "padding stride" as either a compile-time or a run-time value. We also propose adding submdspan (P2630) support for these layouts, and changing P2630 so that submdspan of a layout_left resp. layout_right mdspan produces a layout_left_padded resp. layout_right_padded mdspan whenever possible.

Table of Contents

§ 1. Authors and contributors

§ 1.1. Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)
- Damien Lebrun-Grandie (lebrungrandt@ornl.gov) (Oak Ridge National Laboratory)
- Malte Förster (mfoerster@nvidia.com) (NVIDIA)
- Jiaming Yuan (jiamingy@nvidia.com) (NVIDIA)

§ 2. Revision history

- Revision 0 submitted 2022-09-14
- Revision 1 submitted 2022-10-15

- Change padding stride to function as an overalignment factor if less than the extent to pad. Remove mapping constructor that takes `extents_type` and `extents<index_type, padding_stride>`, because the latter may not be the actual padding stride.
- Make converting constructors from `layout_{left,right}_padded::mapping` to `layout_{left,right}::mapping` use Mandates rather than Constraints to check compile-time stride compatibility.
- Mandate that `layout_{left,right}_padded::mapping`'s actual padding stride, if known at compile time, be representable as a value of type `index_type` (as well as of type `size_t`, the previous requirement).
- Add section explaining why we don't permit conversion from more aligned to less aligned.
- Fixed typos in Wording
- Fix formatting in non-Wording, and add links for BLAS and LAPACK

§ 3. Proposed changes and justification

§ 3.1. Summary of proposed changes

We propose two new mdspan layouts, `layout_left_padded` and `layout_right_padded`. These layouts support two use cases:

1. array layouts that are contiguous in one dimension, as supported by commonly used libraries like the BLAS (Basic Linear Algebra Subroutines; see P1417 and P1674 for historical overview and references) and LAPACK (Linear Algebra PACKage); and
2. "padded" storage for overaligned access of the start of every contiguous segment of the array.

We also propose changing `submdspan` of a `layout_left` resp. `layout_right` mdspan to return `layout_left_padded` resp. `layout_right_padded` instead of `layout_stride`, when the slice arguments permit it.

§ 3.2. Two new mdspan layouts

The two new mdspan layouts `layout_left_padded` and `layout_right_padded` are strided, unique layouts. If the rank is zero or one, then the layouts behave exactly like `layout_left` resp. `layout_right`. If the rank is two or more, then the layouts implement a special case of `layout_stride` where only one stride may differ from the extent that in `layout_left` resp. `layout_right` would completely define the stride. We call that stride the *padding stride*, and the

extent that in `layout_left` resp. `layout_right` would define it the *extent to pad*. The padding stride of `layout_left_padded` is `stride(1)`, and the extent to pad is `extent(0)`. The padding stride of `layout_right_padded` is `stride(rank() - 2)`, and the extent to pad is `extent(rank() - 1)`. All other strides of `layout_left_padded` are the same as in `layout_left`, and all other strides of `layout_right_padded` are the same as in `layout_right`.

§ 3.2.1. Optimizations over `layout_stride`

The two new layouts offer the following optimizations over `layout_stride`.

1. They guarantee at compile time that one extent always has stride-1 access. While `layout_stride`'s member functions are all `constexpr`, its mapping constructor takes the strides as a `std::array` with `rank()` size.
2. They do not need to store any strides if the padding stride is known at compile time. Even if the padding stride is a run-time value, these layouts only need to store the one stride value (as `index_type`). The `layout_stride::mapping` class must store all `rank()` stride values.

§ 3.2.2. New layouts unify two use cases

The proposed layouts unify two different use cases:

1. overaligned access to the beginning of each contiguous segment of elements, and
2. representing exactly the data layout assumed by the General (GE) matrix type in the BLAS' C binding.

Regarding (1), an appropriate choice of padding can ensure any desired overalignment of the beginning of each contiguous segment of elements in an mdspan, as long as the entire memory allocation has the same overalignment. This is useful for hardware features that require or perform better with overaligned access, such as SIMD (Single Instruction Multiple Data) instructions.

Regarding (2), the padding stride is the same as BLAS' "leading dimension" of the matrix (`LDA`) argument. Unlike `layout_left` and `layout_right`, any subview of a contiguous subset of rows and columns of a rank-2 `layout_left_padded` or `layout_right_padded` mdspan preserves the layout. For example, if `A` is a rank-2 mdspan whose layout is `layout_left_padded<padding_stride>`, then `submdspan(A, tuple{r1, r2}, tuple{c1, c2})` also has layout `layout_left_padded<padding_stride>` with the same padding stride as before. The BLAS and algorithms that use it (such as the blocked algorithms in LAPACK) depend on this ability to operate on contiguous submatrices with the same layout as their parent. For this reason,

we can replace the `layout_blas_general` layout in P1673R9 with `layout_left_padded` and `layout_right_padded`. Making most effective use of the new layouts in code that uses P1673 calls for integrating them with `submdspan`. This is why we propose the following changes as well.

3.2.3. Design change from R0

A design change from R0 of this paper makes this overalignment case easier to use and more like the existing `std::assume_aligned` interface. In R0 of this paper, the user's padding input parameter (either a compile-time `padding_stride` or a run-time value) was exactly the padding stride. As such, it had to be greater than or equal to the extent to pad. For example, if users had an `extent(0)` of 13 and wanted to overalign the corresponding `stride(1)` to a multiple of 4, they would have had to specify `layout_left_padded<16>`. This was inconsistent with `std::assume_aligned`, whose template argument (the byte alignment) would need to be `4 * sizeof(element_type)`. Also, users who wanted a compile-time padding stride would have needed to compute it themselves from the corresponding compile-time extent, rather than prespecifying a fixed overalignment factor that could be used for any extent. This was not only harder to use, but it made the layout itself (not just the layout mapping) depend on the extent. That was inconsistent with the existing mdspan layouts, where the layout type itself (e.g., `layout_left`) is always a function from `extents` specialization to layout mapping.

In this version of the paper, we interpret the case where the input padding stride is less than the extent to pad as an "overalignment factor" instead of a stride. To revisit the above example, `layout_left_padded<4>` would take an `extent(0)` of 13 and round up the corresponding `stride(1)` to 16. However, as before, `layout_left_padded<17>` would take an `extent(0)` of 13 and round up the corresponding `stride(1)` to 17. The rule is consistent: the actual padding stride is always the next multiple of the input padding stride greater than or equal to the extent-to-pad.

In R0 of this paper, the following alias

```
using overaligned_matrix_t =
  mdspan<float, dextents<size_t, 2>, layout_right_padded<4>>;
```

would only be meaningful if the run-time extents are less than or equal to 4. In this version of the paper, this alias would always mean "the padding stride rounds up the rightmost extent to a multiple of 4, whatever the extent may be." R0 had no way to express that use case with a compile-time input padding stride. This is important for hardware features and compiler optimizations that require overalignment of multidimensional arrays.

§ 3.2.4. Padding stride equality for layout mapping conversions

`layout_left_padded<padding_stride>::mapping<Extents>` has a converting constructor from `layout_left_padded<other_padding_stride>::mapping<OtherExtents>`. Similarly, `layout_right_padded<padding_stride>::mapping<Extents>` has a converting constructor from `layout_right_padded<other_padding_stride>::mapping<OtherExtents>`. These constructors require, among other conditions, that if `padding_stride` and `other_padding_stride` do not equal `dynamic_extent`, then `padding_stride` equals `other_padding_stride`.

Users may ask why they can't convert a more overaligned mapping, such as `layout_left_padded<4>::mapping`, to a less overaligned mapping, such as `layout_left_padded<2>::mapping`. The problem is that this may not be correct for all extents. For example, the following code would be incorrect if it were well formed (it is not, in this proposal).

```
layout_left_padded<4>::mapping m_orig{extents{9, 2}};
layout_left_padded<2>::mapping m_new(m_orig);
```

The issue is that `m_orig` has an underlying ("physical") layout of `extents{12, 2}`, but `layout_left_padded<2>::mapping{extents{9, 2}}` would have an underlying layout of `extents{10, 2}`. That is, `layout_left_padded<4>::mapping{extents{9, 2}}.stride(1)` is 12, but `layout_left_padded<2>::mapping{extents{9, 2}}.stride(1)` is 10.

In case one is tempted to permit assigning dynamic padding stride to static padding stride, the following code would also be incorrect if it were well formed (it is not, in this proposal). Again, `m_orig.stride(1)` is 12.

```
layout_left_padded<dynamic_extent>::mapping m_orig{extents{9, 2}, 4};
layout_left_padded<2>::mapping m_new(m_orig);
```

The following code is well formed in this proposal, and it gives `m_new` the expected original padding stride of 12.

```
layout_left_padded<dynamic_extent>::mapping m_orig{extents{9, 2}, 4};
layout_left_padded<dynamic_extent>::mapping m_new(m_orig);
```

Similarly, the following code is well formed in this proposal, and it gives `m_new` the expected original padding stride of 12.

```
layout_left_padded<4>::mapping m_orig{extents{9, 2}};
layout_left_padded<dynamic_extent>::mapping m_new(m_orig);
```

§ 3.3. Integration with `submdspan`

We propose changing `submdspan` (see P2630) of a `layout_left` resp. `layout_right` mdspan to return `layout_left_padded` resp. `layout_right_padded` instead of `layout_stride`, if the slice arguments permit it. Taking the `submdspan` of a `layout_left_padded` resp. `layout_right_padded` mdspan will preserve the layout, again if the slice arguments permit it.

The phrase "if the slice arguments permit it" means the following.

§ 3.3.1. `layout_left_padded` and `layout_left` cases

In what follows, let `left_submatrix` be the following function,

```
template<class Elt, class Extents, class Layout, class Accessor, class S0,
requires(
  is_convertible_v<S0,
    tuple<typename Extents::index_type, typename Extents::index_type>> &&
  is_convertible_v<S1,
    tuple<typename Extents::index_type, typename Extents::index_type>>
)
auto left_submatrix(mdspan<Elt, Extents, Layout, Accessor> X, S0 s0, S1 s1)
{
  auto full_extents = []<size_t ... Indices>(index_sequence<Indices...>) {
      return tuple{ (Indices, full_extent)... };
    }(make_index_sequence<X.rank() - 2>());
  return apply( [&](full_extent_t ... fe) {
      return submdspan(X, s0, s1, fe...);
    }, full_extents );
}
```

let `index_type` be an integral type, let `s0` be an object of a type `S0` such that `is_convertible_v<S0, tuple<index_type, index_type>>` is `true`, and let `s1` be an object of a type `S1` such that `is_convertible_v<S1, tuple<index_type, index_type>>` is `true`.

Let `X` be an `mdspan` with rank at least two with `decltype(X)::index_type` naming the same type as `index_type`, whose layout is `layout_left_padded<padding_stride_X>` for some `constexpr size_t padding_stride_X`. Let `X_sub` be the object returned from `left_submatrix(X, s0, s1)`. Then, `X_sub` is an `mdspan` of rank `X.rank()` with layout `layout_left_padded<padding_stride_X>`, and `X_sub.stride(1)` equals `X.stride(1)`.

Let `Z` be an `mdspan` with rank at least two with `decltype(Z)::index_type` naming the same type as `index_type`, whose layout is `layout_left`. Let `Z_sub` be the object returned from `left_submatrix(Z, s0, s1)`. Then, `Z_sub` is an `mdspan` of rank `Z.rank()` with layout `layout_left_padded<padding_stride_Z>`, where `padding_stride_Z` is

- `srm1_val1 - srm1_val0`, if `srm1` is convertible to `tuple<integral_constant<decltype(W)::index_type, srm1_val0>, integral_constant<decltype(W)::index_type, srm1_val1>>` with `srm1_val1` greater than to equal to `srm1_val0`; else,
- `dynamic_rank`.

Also, `Z_sub.stride(1)` equals `Z.stride(1)`.

3.3.2. `layout_right_padded` and `layout_right` cases

In what follows, let `right_submatrix` be the following function,

```
template<class Elt, class Extents, class Layout, class Accessor, class Srm2
requires(
  is_convertible_v<Srm2,
    tuple<typename Extents::index_type, typename Extents::index_type>> &&
  is_convertible_v<Srm1,
    tuple<typename Extents::index_type, typename Extents::index_type>>
)
auto left_submatrix(mdspan<Elt, Extents, Layout, Accessor> X, Srm2 srm2, Sr
{
  auto full_extents = []<size_t ... Indices>(index_sequence<Indices...>) {
      return tuple{ (Indices, full_extent)... };
    }(make_index_sequence<X.rank() - 2>());
  return apply( [&](full_extent_t ... fe) {
```

```
      return submdspan(X, fe..., srm2, srm1);
    }, full_extents );
}
```

let `srm2` ("s of rank minus 2") be an object of a type `Srm2` such that `is_convertible_v<S0, tuple<index_type_X, index_type_X>>` is `true`, and let `srm1` ("s of rank minus 1") be an object of a type `Srm1` such that `is_convertible_v<S1, tuple<index_type_X, index_type_X>>` is `true`.

Similarly, let `Y` be an `mdspan` with rank at least two whose layout is `layout_right_padded<padding_stride_Y>` for some `constexpr size_t padding_stride_Y`. Let `index_type_Y` name the type `decltype(Y)::index_type`. Let `srm2` ("S of rank minus 2") be an object of a type `Srm2` such that `is_convertible_v<Srm2, tuple<index_type_Y, index_type_Y>>` is `true`, and let `srm1` ("S of rank minus 1") be an object of a type `Srm1` such that `is_convertible_v<Srm1, tuple<index_type_Y, index_type_Y>>` is `true`. In the following code fragment,

```
auto full_extents = []<size_t ... Indices>(index_sequence<Indices...>) {
    return tuple{ (Indices, full_extent)... };
  }(make_index_sequence<Y.rank() - 2>());

auto Y_sub = apply( [&](full_extent_t ... fe) {
    return submdspan(Y, fe..., srm2, srm1);
  }, full_extents );
```

`Y_sub` is an `mdspan` of rank `Y.rank()` with layout `layout_left_padded<padding_stride>`, and `Y_sub.stride(1)` equals `Y.stride(1)`.

Let `Z` be an `mdspan` with rank at least two whose layout is `layout_left`. Let `index_type_Z` name the type `decltype(Z)::index_type`. Let `s0` be an object of a type `S0` such that `is_convertible_v<S0, tuple<index_type_Z, index_type_Z>>` is `true`, and let `s1` be an object of a type `S1` such that `is_convertible_v<S1, tuple<index_type_Z, index_type_Z>>` is `true`. In the following code fragment,

```
auto full_extents = []<size_t ... Indices>(index_sequence<Indices...>) {
    return tuple{ (Indices, full_extent)... };
  }(make_index_sequence<Z.rank() - 2>());
```

```
auto Z_sub = apply( [&](full_extent_t ... fe) {
    return submdspan(Z, s0, s1, fe...);
  }, full_extents );
```

`Z_sub` is an `mdspan` of rank `Z.rank()` with layout `layout_left_padded<padding_stride_Z>`, where `padding_stride_Z` is `s0_val1 - s0_val0` if `s0` is convertible to `tuple<integral_constant<index_type_Z, s0_val0>, integral_constant<index_type_Z, s0_val1>>` with `s0_val1` greater than to equal to `s0_val0`. Also, `Z_sub.stride(1)` equals `Z.stride(1)`.

Similarly, let `W` be an `mdspan` with rank at least two whose layout is `layout_right`. Let `index_type_W` name the type `decltype(W)::index_type`. Let `srm2` ("S of rank minus 2") be an object of a type `Srm2` such that `is_convertible_v<Srm2, tuple<index_type_W, index_type_W>>` is `true`, and let `srm1` ("S of rank minus 1") be an object of a type `Srm1` such that `is_convertible_v<Srm1, tuple<index_type_W, index_type_W>>` is `true`. In the following code fragment,

```
auto full_extents = []<size_t ... Indices>(index_sequence<Indices...>) {
    return tuple{ (Indices, full_extent)... };
  }(make_index_sequence<W.rank() - 2>());

auto W_sub = apply( [&](full_extent_t ... fe) {
    return submdspan(W, fe..., srm2, srm1);
  }, full_extents );
```

`W_sub` is an `mdspan` of rank `W.rank()` with layout `layout_left_padded<padding_stride_W>`, where `padding_stride_W` is `srm1_val1 - srm1_val0` if `srm1` is convertible to `tuple<integral_constant<index_type_W, srm1_val0>, integral_constant<index_type_W, srm1_val1>>` with `srm1_val1` greater than to equal to `srm1_val0`. Also, `W_sub.stride(1)` equals `W.stride(1)`.

Preservation of these layouts under `submdspan` is an important feature for our linear algebra library proposal P1673, because it means that for existing BLAS and LAPACK use cases, if we start with one of these layouts, we know that we can implement fast linear algebra algorithms by calling directly into an optimized C or Fortran BLAS.

§ 3.4. Examples

§ 3.4.1. Directly call C BLAS without checks

We show examples before and after this proposal of functions that compute the matrix-matrix product `C += A * B`. The `recursive_matrix_product` function computes this product recursively, by partitioning each of the three matrices into a 2 x 2 block matrix using the `partition` function. When the `C` matrix is small enough, `recursive_matrix_product` stops recursing and instead calls a `base_case_matrix_product` function with different overloads for different matrix layouts. If the matrix layouts support it, `base_case_matrix_product` can call the C BLAS function `cblas_sgemm` directly on the `mdspan`s' data. This is fast if the C BLAS is optimized. Otherwise, `base_case_matrix_product` falls back to a slow generic implementation.

This example is far from ideally optimized, but it hints at the kind of optimizations that linear algebra computations do in practice.

Common code:

```
template<class Layout>
using out_matrix_view = mdspan<float, dextents<int, 2>, Layout>;

template<class Layout>
using in_matrix_view = mdspan<const float, dextents<int, 2>, Layout>;

// Before this proposal, if Layout is layout_left or layout_right,
// the returned mdspan would all be layout_stride.
// After this proposal, the returned mdspan would be
// layout_left_padded resp. layout_right_padded.
template<class ElementType, class Layout>
auto partition(mdspan<ElementType, dextents<int, 2>, Layout> A)
{
  auto M = A.extent(0);
  auto N = A.extent(1);
  auto A00 = submdspan(A, tuple{0, M / 2}, tuple{0, N / 2});
  auto A01 = submdspan(A, tuple{0, M / 2}, tuple{N / 2, N});
  auto A10 = submdspan(A, tuple{M / 2, M}, tuple{0, N / 2});
  auto A11 = submdspan(A, tuple{M / 2, M}, tuple{N / 2, N});
  return tuple{
    A00, A01,
    A10, A11
  };
}
```

```
template<class Layout>
void recursive_matrix_product(in_matrix_view<Layout> A,
  in_matrix_view<Layout> B, out_matrix_view<Layout> C)
{
  // Some hardware-dependent constant
  constexpr int recursion_threshold = 16;
  if(std::max(C.extent(0) || C.extent(1)) <= recursion_threshold) {
    base_case_matrix_product(A, B, C);
  } else {
    auto [C00, C01,
          C10, C11] = partition(C);
    auto [A00, A01,
          A10, A11] = partition(A);
    auto [B00, B01,
          B10, B11] = partition(B);
    recursive_matrix_product(A00, B00, C00);
    recursive_matrix_product(A01, B10, C00);
    recursive_matrix_product(A10, B00, C10);
    recursive_matrix_product(A11, B10, C10);
    recursive_matrix_product(A00, B01, C01);
    recursive_matrix_product(A01, B11, C01);
    recursive_matrix_product(A10, B01, C11);
    recursive_matrix_product(A11, B11, C11);
  }
}

// Slow generic implementation
template<class Layout>
void base_case_matrix_product(in_matrix_view<Layout> A,
  in_matrix_view<Layout> B, out_matrix_view<Layout> C)
{
  for(size_t j = 0; j < C.extent(1); ++j) {
    for(size_t i = 0; i < C.extent(0); ++i) {
      typename out_matrix_view<Layout>::value_type C_ij{};
      for(size_t k = 0; k < A.extent(1); ++k) {
        C_ij += A(i,k) * B(k,j);
      }
      C(i,j) += C_ij;
    }
  }
}
```

A user might interpret `layout_left` as "column major," and therefore "the natural layout to pass into the BLAS."

```
void base_case_matrix_product(in_matrix_view<layout_left> A,
  in_matrix_view<layout_left> B, out_matrix_view<layout_left> C)
{
  cblas_sgemm(CblasColMajor, CblasNoTrans, CblasNoTrans,
    C.extent(0), C.extent(1), A.extent(1), 1.0f,
    A.data_handle(), A.stride(1), B.data_handle(), B.stride(1),
    1.0f, C.data_handle(), C.stride(1));
}
```

However, `recursive_matrix_product` never gets to use the `layout_left` overload of `base_case_matrix_product`, because the base case matrices are always `layout_stride`.

On discovering this, the author of these functions might be tempted to write a custom layout for "BLAS-compatible" matrices. However, the `submdspan` proposal P2630R0 currently forces `partition` to return four `layout_stride` mdspan if given a `layout_left` (or `layout_right`) input mdspan. This would, in turn, force users of `recursive_matrix_product` to commit to a custom layout, if they want to use the BLAS.

Alternately, the author of these functions could specialize `base_case_matrix_product` for `layout_stride`, and check whether `A.stride(0)`, `B.stride(0)`, and `C.stride(0)` are all equal to one before calling `cblas_sgemm`. However, that would force extra run-time checks for a use case that most users might never encounter, because most users are starting with `layout_left` matrices or contiguous submatrices thereof.

After our proposal, the author can specialize `base_case_matrix_product` for exactly the layout supported by the BLAS. They could even get rid of the fall-back implementation if users never exercise it.

```
template<size_t p>
void base_case_matrix_product(in_matrix_view<layout_left_padded<p>> A,
  in_matrix_view<layout_left_padded<p>> B,
  out_matrix_view<layout_left_padded<p>> C)
{ // same code as above
  cblas_sgemm(CblasColMajor, CblasNoTrans, CblasNoTrans,
    C.extent(0), C.extent(1), A.extent(1), 1.0f,
```

```
    A.data_handle(), A.stride(1), B.data_handle(), B.stride(1),
    1.0f, C.data_handle(), C.stride(1));
}
```

§ 3.4.2. Overaligned access

By combining these new layouts with an accessor that ensures overaligned access, we can create an mdspan for which the beginning of every contiguous segment of elements is overaligned by some given factor. This can enable use of hardware features that require overaligned memory access.

The following `aligned_accessor` class template (which this proposal does *not* propose to add to the C++ Standard Library) uses the C++ Standard Library function `assume_aligned` to decorate pointer access.

```
template<class ElementType, std::size_t byte_alignment>
struct aligned_accessor {
  // Even if a pointer p is aligned, p + i might not be.
  using offset_policy = std::default_accessor<ElementType>;

  using element_type = ElementType;
  using reference = ElementType&;
  // Some implementations might have an easier time optimizing
  // if this class applies an attribute to the pointer type.
  // Examples of attributes include
  // __declspec(align_value(byte_alignment))
  // and
  // __attribute__((align_value(byte_alignment))).
  using data_handle_type = ElementType*;

  constexpr aligned_accessor() noexcept = default;

  // A feature of default_accessor that permits
  // conversion from nonconst to const.
  template<class OtherElementType, std::size_t other_byte_alignment>
  requires (
    std::is_convertible_v<OtherElementType(*)[], element_type(*)[]> &&
    other_byte_alignment == byte_alignment)
  constexpr aligned_accessor(
    aligned_accessor<OtherElementType, other_byte_alignment>) noexcept
  {}
```

```
  constexpr reference
  access(data_handle_type p, size_t i) const noexcept {
    return std::assume_aligned< byte_alignment >(p)[i];
  }

  constexpr typename offset_policy::data_handle_type
  offset(data_handle_type p, size_t i) const noexcept {
    return p + i;
  }
};
```

We include some helper functions for making overaligned array allocations.

```
template<class ElementType>
struct delete_raw {
  void operator()(ElementType* p) const {
    std::free(p);
  }
};

template<class ElementType>
using allocation_t =
  std::unique_ptr<ElementType[], delete_raw<ElementType>>;

template<class ElementType, std::size_t byte_alignment>
allocation_t<ElementType>
allocate_raw(const std::size_t num_elements)
{
  const std::size_t num_bytes = num_elements * sizeof(ElementType);
  void* ptr = std::aligned_alloc(byte_alignment, num_bytes);
  return {ptr, delete_raw<ElementType>{}};
}
```

Now we can show our example. This 15 x 17 matrix of `float` will have extra padding so that every column is aligned to `8 * sizeof(float)` bytes. We can use the layout mapping to determine the required storage size (including padding). Users can then prove at compile time that they can use

special hardware features that require overaligned access and/or assume that the padding element at the end of each column is accessible memory.

```
constexpr std::size_t element_alignment = 8;
constexpr std::size_t byte_alignment = element_alignment * sizeof(float);

using layout_type = layout_left_padded<element_alignment>;
layout_type::mapping mapping{dextents<int, 2>{15, 17}};
auto allocation =
  allocate_raw<float, byte_alignment>(mapping.required_span_size());

using accessor_type = aligned_accessor<float, byte_alignment>;
mdspan m{allocation.get(), mapping, accessor_type{}};

// m_sub has the same layout as m,
// and each column of m_sub has the same overalignment.
auto m_sub = submdspan(m, tuple{0, 11}, tuple{1, 13});
```

§ 3.5. Alternatives

We considered a variant of `layout_stride` that could encode any combination of compile-time or run-time strides in the layout type. This could, for example, use the same mechanism that `extents` uses. (The reference implementation calls this mechanism a "partially static array.") However, we rejected this approach as overly complex for our design goals.

First, the goal of `layout_{left,right}_padded` isn't to insist even harder that the compiler bake constants into `mapping::operator()` evaluation. The goal is to communicate compile-time information to *users*. The most benefit comes not just from knowing the padding stride at compile time, but also from knowing that one dimension always uses stride-one (contiguous) storage. Putting these two pieces of information together lets users apply compiler annotations like `assume_aligned`, as in the above `aligned_accessor` example. Knowing that one dimension always uses contiguous storage also tells users that they can pass the mdspan's data directly into C or Fortran libraries like the BLAS or LAPACK. Users can benefit from this even if the padding stride is a run-time value.

Second, the `constexpr` annotations in the existing layout mappings mean that users might be evaluating `layout_stride::mapping::operator()` fully at compile time. The reference mdspan implementation has several tests that demonstrate this by using the result of a layout mapping evaluation in a context where it needs to be known at compile time.

Third, the performance benefit of storing some strides as compile-time constants goes down as the rank increases, because most of the strides would end up depending on run-time values anyway. Strided mdspan generally come from a subview of an existing `layout_left` or `layout_right` mdspan. In that case, the representation of the strides that preserves the most compile-time information would be just the original mdspan's `extents_type` object. (Compare to the exposition-only `inner-mapping_` which we use in the wording for `layout_{left,right}_padded`.) Computing each stride would then call for a forward (for `layout_left`) or reverse (for `layout_right`) product of the original mdspan's extents. As a result, any stride to the right resp. left of a run-time extent would end up depending on that run-time extent anyway. The larger the rank, the more strides get "touched" by run-time information.

Fourth, a strided mdspan that can represent layouts as general as `layout_stride`, but has entirely compile-time extents *and* strides, could be useful for supporting features of a specific computer architecture. However, these hardware features would probably have limitations that would prevent them from supporting general strided layouts anyway. For example, they might require strides to be a power of two, or they might be limited to specific ranges of extents or strides. These limitations would call for custom implementation-specific layouts, not something as general as a "compile-time `layout_stride`."

§ 3.6. Implementation experience

Pull request 180 in the reference mdspan implementation implements most of this proposal. Next steps are to add constructors to the existing layout mappings, and to add `submdspan` support for the new layouts.

§ 3.7. Desired ship vehicle

C++26 / IS.

§ 4. Wording

> Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The `�` character is used to denote a placeholder section number which the editor shall determine. First, apply all wording from P2630R0. (This proposal is a "rebase" atop the changes proposed by P2630R0.)

> Add the following feature test macro to *[version.syn]*, replacing YYYYMML with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_mdspan_layout_padded YYYYMML // also in <mdspan>
```

> In Section 🔲 *[mdspan.syn]*, after `struct layout_stride;`, add the following:

```
template<size_t padding_stride = dynamic_extent>
struct layout_left_padded;
template<size_t padding_stride = dynamic_extent>
struct layout_right_padded;
```

> In Section 🔲 *[mdspan.layout.left.overview]* ("Overview"), add the following constructor to the `layout_left::mapping` class declaration, between the constructor converting from `layout_right::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`:

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
    mapping(const layout_left_padded<other_padding_stride>::mapping<OtherEx
```

> In Section 🔲 *[mdspan.layout.left.cons]* ("Constructors"), add the following between the constructor converting from `layout_right::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`:

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
    mapping(const layout_left_padded<other_padding_stride>::mapping<OtherEx
```

Constraints: `is_constructible_v<extents_type, OtherExtents>` is `true`.

Mandates: If

- `Extents::rank()` is greater than one,

- `Extents::static_extent(0)` does not equal `dynamic_extent`,
- `OtherExtents::static_extent(0)` does not equal `dynamic_extent`, and
- `other_padding_stride` does not equal `dynamic_extent`,

then the least multiple of `other_padding_stride` greater than or equal to `OtherExtents::static_extent(0)`

- is representable as a value of type `Extents::index_type`, and
- equals `Extents::static_extent(0)`.

Preconditions:

- If `OtherExtents::rank()` is greater than one, then `other.stride(1)` is representable as a value of type `index_type`;
- if `extents_type::rank() > 0` is `true`, then for all r in the range [0, `extents_type::rank()`), `other.stride( $r$ )` equals `extents().fwd-prod-of-extents( $r$ )`, and
- `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

Effects: Direct-non-list-initializes `extents_` with `other.extents()`.

> In Section 🔲 *[mdspan.layout.right.overview]* ("Overview"), add the following constructor to the `layout_right::mapping` class declaration, between the constructor converting from `layout_left::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`:

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
    mapping(const layout_right_padded<other_padding_stride>::mapping<OtherE
```

> In Section 🔲 *[mdspan.layout.right.cons]* ("Constructors"), add the following between the constructor converting from `layout_left::mapping<OtherExtents>` and the constructor converting from `layout_stride::mapping<OtherExtents>`:

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
    mapping(const layout_right_padded<other_padding_stride>::mapping<OtherE
```

Constraints: `is_constructible_v<extents_type, OtherExtents>` is `true`.

Mandates: If

- `Extents::rank()` is greater than one,
- `Extents::static_extent(Extents::rank() - 1)` does not equal `dynamic_extent`,
- `OtherExtents::static_extent(Extents::rank() - 1)` does not equal `dynamic_extent`, and
- `other_padding_stride` does not equal `dynamic_extent`,

then the least multiple of `other_padding_stride` greater than or equal to `OtherExtents::static_extent(OtherExtents::rank() - 1)`

- is representable as a value of type `Extents::index_type`, and
- equals `Extents::static_extent(Extents::rank() - 1)`.

Preconditions:

- If `OtherExtents::rank()` is greater than one, then `other.stride(OtherExtents::rank() - 2)` is representable as a value of type `index_type`;
- if `extents_type::rank() > 0` is `true`, then for all r in the range [0, `extents_type::rank()`), `other.stride( $r$ )` equals `extents().rev-prod-of-extents( $r$ )`, and
- `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

Effects: Direct-non-list-initializes `extents_` with `other.extents()`.

After the end of Section 🔲 *[mdspan.layout.stride]*, add the following:

§ 4.1. Class template `layout_left_padded::mapping` [mdspan.layout.left_padded]

`layout_left_padded` provides a layout mapping that behaves like `layout_left::mapping`, except that the *padding stride* `stride(1)` can be greater than or equal to `extent(0)`. Users provide an input padding stride value either as a `size_t` template parameter `padding_stride` of `layout_left_padded`, or as a run-time argument of `layout_left_padded::mapping`'s constructor. The padding stride is the least multiple of the input padding stride value greater than or equal to `extent(0)`.

```
template<size_t padding_stride = dynamic_extent>
struct layout_left_padded {
  template<class Extents>
  class mapping {
  public:
    using extents_type = Extents;
    using index_type = typename extents_type::index_type;
    using size_type = typename extents_type::size_type;
    using rank_type = typename extents_type::rank_type;
    using layout_type = layout_left_padded<padding_stride>;

  private:
    static constexpr size_t <it>actual-padding-stride</it> = /* see-below *

    using <it>inner-extents-type</it> = /* see-below */; // exposition only
    using <it>unpadded-extent-type</it> = /* see-below */; // exposition or
    using <it>inner-mapping-type</it> =
      layout_left::template mapping<<it>inner-extents-type</it>>; // exposi

    <it>inner-mapping-type</it> <it>inner-mapping_</it>; // exposition only
    <it>unpadded-extent-type</it> <it>unpadded-extent_</it>; // exposition

  public:
    constexpr mapping(const extents_type& ext);

    template<class Size>
    constexpr mapping(const extents_type& ext, Size padding_value);
```

```
    template<size_t other_padding_stride, class OtherExtents>
      constexpr explicit( /* see below */ )
        mapping(const layout_left_padded<other_padding_stride>::mapping<Oth

    template<size_t other_padding_stride, class OtherExtents>
      constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
        mapping(const layout_right_padded<other_padding_stride>::mapping<Ot

    constexpr mapping(const mapping&) noexcept = default;
    mapping& operator=(const mapping&) noexcept = default;

    constexpr extents_type extents() const noexcept;

    constexpr std::array<index_type, extents_type::rank()>
    strides() const noexcept;

    constexpr index_type required_span_size() const noexcept;

    template<class... Indices>
    constexpr size_t operator()(Indices... idxs) const noexcept;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_exhaustive() noexcept;
    static constexpr bool is_always_strided() noexcept { return true; }

    static constexpr bool is_unique() noexcept { return true; }
    constexpr bool is_exhaustive() const noexcept;
    static constexpr bool is_strided() noexcept { return true; }

    constexpr index_type stride(rank_type r) const noexcept;
  };
};
```

Throughout this section, let `P_left` be the following size `extents_type::rank()` parameter pack of `size_t`:

- If `extents_type::rank()` equals zero or one, then the empty parameter pack;
- else, the parameter pack `size_t(1)`, `size_t(2)`, ..., `extents_type::rank() - 1`.

Mandates: If

- `extents_type::rank()` is greater than one,

- `padding_stride` does not equal `dynamic_extent`, and
- `extents_type::static_extent(0)` does not equal `dynamic_extent`,

then the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(0)` is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

```
static constexpr size_t <it>actual-padding-stride</it> = /* see-below */; /
```

- If `extents_type::rank()` equals zero or one, then `padding_stride`.
- Else, if
  - `padding_stride` does not equal `dynamic_extent` and
  - `extents_type::static_extent(0)` does not equal `dynamic_extent`,

  then the `size_t` value which is the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(0)`.
- Otherwise, `dynamic_extent`.

```
using <it>inner-extents-type</it> = /* see-below */; // exposition only
```

- If `extents_type::rank()` equals zero or one, then `inner-extents-type` names the type `extents_type`.
- Otherwise, `inner-extents-type` names the type `extents<index_type, actual-padding-stride , extents_type::static_extent(P_left)...>`.

```
using <it>unpadded-extent-type</it> = /* see-below */; // exposition only
```

- If `extents_type::rank()` equals zero, then `unpadded-extent-type` names the type `extents<index_type>`.
- Otherwise, `unpadded-extent-type` names the type `extents<index_type, extents_type::static_extent(0)>`.

```
constexpr mapping(const extents_type& ext);
```

Preconditions: If `extents_type::rank()` is greater than one and `padding_stride` does not equal `dynamic_extent`, then the least multiple of `padding_stride` greater than to equal to `ext.extent(0)` is representable as a value of type `index_type`.

Effects:

- Direct-non-list-initializes `inner-mapping_` with:
  - `ext`, if `extents_type::rank()` is zero or one; else,
  - `ext.extent(0), ext.extent(P_left)...`, if `padding_stride` is `dynamic_extent`; else,
  - `S_left, ext.extent(P_left)...`, where `S_left` is the least multiple of `padding_stride` greater than or equal to `ext.extent(0)`; and
- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, direct-non-list-initializes `unpadded-extent_` with `ext.extent(0)`.

```
template<class Size>
constexpr mapping(const extents_type& ext, Size padding_value);
```

Constraints:

- `is_convertible_v<Size, index_type>` is `true`, and
- `is_nothrow_constructible_v<index_type, Size>` is `true`.

Preconditions:

- If `padding_stride` does not equal `dynamic_extent`, then
  - `padding_value` is representable as a value of type `index_type`, and
  - the result of converting `padding_value` to `index_type` equals `padding_stride`.
- If `extents_type::rank()` is greater than one, then the least multiple of `padding_value` greater than to equal to `ext.extent(0)` is representable as a value of type `index_type`.

Effects:

- Direct-non-list-initializes `inner-mapping_` with:
  - `ext`, if `extents_type::rank()` is zero or one; else,
  - `S_left, ext.extent(P_left)...`, where `S_left` is the least multiple of `padding_value` greater than or equal to `ext.extent(0)`; and

- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, direct-non-list-initializes `unpadded-extent_` with `ext.extent(0)`.

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit( /* see below */ )
    mapping(const layout_left_padded<other_padding_stride>::mapping<OtherEx
```

Constraints: `is_constructible_v<extents_type, OtherExtents>` is `true`.

Mandates: `padding_stride == dynamic_extent || other_padding_stride == dynamic_extent || padding_stride == other_padding_stride` is `true`.

Preconditions:

- If `extents_type::rank() > 1` is `true` and `padding_stride` does not equal `dynamic_extent`, then `other.stride(1)` equals the least multiple of `padding_stride` greater than or equal to `extents_type:: index-cast (other.extent(0))`; and
- `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

Effects:

- Direct-non-list-initializes `inner-mapping_` with:
  - `other.extents()`, if `extents_type::rank()` is zero or one; else
  - `other.stride(1), other.extents().extent(P_left)...`; and
- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, direct-non-list-initializes `unpadded-extent_` with `other.extents().extent(0)`.

Remarks: The expression inside `explicit` is equivalent to: `extents_type::rank() > 1 && (padding_stride == dynamic_extent || other_padding_stride == dynamic_extent)`.

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
    mapping(const layout_right_padded<other_padding_stride>::mapping<OtherE
```

Constraints:

- `extents_type::rank()` equals zero or one,
- `is_constructible_v<extents_type, OtherExtents>` is `true`.

Precondition: `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

Effects:

- Direct-non-list-initializes `inner-mapping_` with `other.extents()`; and
- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, direct-non-list-initializes `unpadded-extent_` with `other.extents().extent(0)`.

[Note: Neither mapping uses the padding stride in the rank-0 or rank-1 case, so the padding stride does not affect either the constraints or the preconditions. — *end note]*

```
constexpr extents_type extents() const noexcept;
```

Effects:

- If `extents_type::rank()` is zero, equivalent to `return extents_type{};`.
- Otherwise, equivalent to `return extents_type( unpadded-extent_ .extent(0), inner-mapping_ .extent(P_left)...);`.

```
constexpr std::array<index_type, extents_type::rank()>
  strides() const noexcept;
```

Effects: Equivalent to `return inner-mapping_ .strides();`.

```
constexpr index_type required_span_size() const noexcept;
```

Effects: Equivalent to `return inner-mapping_ .required_span_size();`.

```
template<class... Indices>
constexpr size_t operator()(Indices... idxs) const noexcept;
```

Constraints:

- `sizeof...(Indices) == Extents::rank()` is `true`,

- `(std::is_convertible_v<Indices, index_type> && ...)` is `true`, and
- `(std::is_nothrow_constructible<index_type, Indices> && ...)` is `true`.

Precondition: `extents_type::` `index-cast` `(i)` is a multidimensional index in `extents()` ([mdspan.overview]).

Effects: Let P be a parameter pack such that `is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>` is `true`. Equivalent to: `return ((static_cast<index_type>(i) * stride(P)) + ... + 0);`.

[Note: Effects are also equivalent to

`return inner-mapping_ (idxs...);`, but only after the Precondition has been applied. — *end note]*

```
static constexpr bool is_always_exhaustive() noexcept;
```

Returns:

- If `extents_type::rank()` equals zero or one, then `true`;
- else, if neither `<it>inner-mapping-type</it>::static_extent(0)` nor `extents_type::static_extent(0)` equal `dynamic_extent`, then `<it>inner-mapping-type</it>::static_extent(0) == extents_type::static_extent(0)`;
- otherwise, `false`.

```
constexpr bool is_exhaustive() const noexcept;
```

Returns:

- If `extents_type::rank()` equals zero, then `true`;
- else, `inner-mapping_ .extent(0) == unpadded-extent_ .extent(0)`.

```
constexpr index_type stride(rank_type r) const noexcept;
```

Effects: Equivalent to `return inner-mapping_ .stride(r);`.

§ 4.2. Class template `layout_right_padded::mapping` [mdspan.layout.right_padded]

`layout_right_padded` provides a layout mapping that behaves like `layout_right::mapping`, except that the *padding stride* `stride(rank() - 2)` can be greater than or equal to `extent(rank() - 1)`. Users provide an input padding stride value either as a `size_t` template parameter `padding_stride` of `layout_right_padded`, or as a run-time argument of `layout_right_padded::mapping`'s constructor. The padding stride is the least multiple of the input padding stride value greater than or equal to `extent(rank() - 1)`.

```
template<size_t padding_stride = dynamic_extent>
struct layout_right_padded {
  template<class Extents>
  struct mapping {
  public:
    using extents_type = Extents;
    using index_type = typename extents_type::index_type;
    using size_type = typename extents_type::size_type;
    using rank_type = typename extents_type::rank_type;
    using layout_type = layout_right_padded<padding_stride>;

  private:
    static constexpr size_t <it>actual-padding-stride</it> = /* see-below *

    using <it>inner-extents-type</it> = /* see-below */; // exposition only
    using <it>unpadded-extent-type</it> = /* see-below */; // exposition or
    using <it>inner-mapping-type</it> =
      layout_right::template mapping<<it>inner-extents-type</it>>; // expos

    <it>inner-mapping-type</it> <it>inner-mapping_</it>; // exposition only
    <it>unpadded-extent-type</it> <it>unpadded-extent_</it>; // exposition

  public:
    constexpr mapping(const extents_type& ext);

    template<class Size>
    constexpr mapping(const extents_type& ext, Size padding_value);

    template<size_t other_padding_stride, class OtherExtents>
      constexpr explicit( /* see below */ )
        mapping(const layout_right_padded<other_padding_stride>::mapping<Ot
```

```
    template<size_t other_padding_stride, class OtherExtents>
      constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
        mapping(const layout_left_padded<other_padding_stride>::mapping<Oth

    constexpr mapping(const mapping&) noexcept = default;
    mapping& operator=(const mapping&) noexcept = default;

    constexpr extents_type extents() const noexcept;

    constexpr std::array<index_type, extents_type::rank()>
    strides() const noexcept;

    constexpr index_type required_span_size() const noexcept;

    template<class... Indices>
    constexpr size_t operator()(Indices... idxs) const noexcept;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_exhaustive() noexcept;
    static constexpr bool is_always_strided() noexcept { return true; }

    static constexpr bool is_unique() noexcept { return true; }
    constexpr bool is_exhaustive() const noexcept;
    static constexpr bool is_strided() noexcept { return true; }

    constexpr index_type stride(rank_type r) const noexcept;
  };
};
```

Throughout this section, let `P_right` be the following size `extents_type::rank()` parameter pack of `size_t`:

- If `extents_type::rank()` equals zero or one, then the empty parameter pack;
- else, the parameter pack `size_t(0)`, `size_t(1)`, ..., `extents_type::rank() - 2`.

Mandates: If

- `extents_type::rank()` is greater than one,
- `padding_stride` does not equal `dynamic_extent`, and
- `extents_type::static_extent(extents_type::rank() - 1)` does not equal `dynamic_extent`,

then the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(extents_type::rank() - 1)` is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

```
static constexpr size_t <it>actual-padding-stride</it> = /* see-below */; /
```

- If `extents_type::rank()` equals zero or one, then `padding_stride`.
- Else, if
  - `padding_stride` does not equal `dynamic_extent` and
  - `extents_type::static_extent(0)` does not equal `dynamic_extent`,

  then the `size_t` value which is the least multiple of `padding_stride` that is greater than or equal to `extents_type::static_extent(0)`.
- Otherwise, `dynamic_extent`.

```
using <it>inner-extents-type</it> = /* see-below */; // exposition only
```

- If `extents_type::rank()` equals zero or one, then `inner-extents-type` names the type `extents_type`.
- Otherwise, `inner-extents-type` names the type `extents<index_type, extents_type::static_extent(P_right)..., actual-padding-stride >`.

```
using <it>unpadded-extent-type</it> = /* see-below */; // exposition only
```

- If `extents_type::rank()` equals zero, then `unpadded-extent-type` names the type `extents<index_type>`.
- Otherwise, `unpadded-extent-type` names the type `extents<index_type, extents_type::static_extent(Extents::rank() - 1)>`.

```
constexpr mapping(const extents_type& ext);
```

Preconditions: If `extents_type::rank()` is greater than one and `padding_stride` does not equal `dynamic_extent`, then the least multiple of `padding_stride` greater than to equal to `ext.extent(extents_type::rank() - 1)` is representable as a value of type `index_type`.

Effects:

- Direct-non-list-initializes `inner-mapping_` with:
  - `ext`, if `extents_type::rank()` is zero or one; else,
  - `ext.extent(P_right)..., ext.extent(extents_type::rank() - 1)`, if `padding_stride` is `dynamic_extent`; else,
  - `ext.extent(P_right)..., S_right`, where `S_right` is the least multiple of `padding_stride` greater than or equal to `ext.extent(extents_type::rank() - 1)`; and
- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, direct-non-list-initializes `unpadded-extent_` with `ext.extent(extents_type::rank() - 1)`.

```
template<class Size>
constexpr mapping(const extents_type& ext, Size padding_value);
```

Constraints:

- `is_convertible_v<Size, index_type>` is `true`, and
- `is_nothrow_constructible_v<index_type, Size>` is `true`.

Preconditions:

- If `padding_stride` does not equal `dynamic_extent`, then
  - `padding_value` is representable as a value of type `index_type`, and
  - the result of converting `padding_value` to `index_type` equals `padding_stride`.
- If `extents_type::rank()` is greater than one, then the least multiple of `padding_value` greater than to equal to `ext.extent(extents_type::rank() - 1)` is representable as a value of type `index_type`.

Effects:

- Direct-non-list-initializes `inner-mapping_` with:
  - `ext`, if `extents_type::rank()` is zero or one; else
  - `ext.extent(P_right)..., S_right`, where `S_right` is the least multiple of `padding_value` greater than or equal to `ext.extent(extents_type::rank() - 1)`; and

- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, direct-non-list-initializes `unpadded-extent_` with `ext.extent(extents_type::rank() - 1)`.

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit( /* see below */ )
    mapping(const layout_right_padded<other_padding_stride>::mapping<OtherE
```

Constraints: `is_constructible_v<extents_type, OtherExtents>` is `true`.

Mandates: `padding_stride == dynamic_extent || other_padding_stride == dynamic_extent || padding_stride == other_padding_stride` is `true`.

Preconditions:

- If `extents_type::rank() > 1` is `true` and `padding_stride` does not equal `dynamic_extent`, then `other.stride(extents_type::rank() - 2)` equals the least multiple of `padding_stride` greater than or equal to `extents_type:: index-cast (other.extent(OtherExtents::rank() - 1))`; and
- `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

Effects:

- Direct-non-list-initializes `inner-mapping_` with:
  - `other.extents()`, if `extents_type::rank()` is zero or one; else,
  - `other.extents().extent(P_right)...`, `other.stride(extents_type::rank() - 2)`; and
- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, direct-non-list-initializes `unpadded-extent_` with `other.extents().extent(extents_type::rank() - 1)`.

Remarks: The expression inside `explicit` is equivalent to: `extents_type::rank() > 1 && (padding_stride == dynamic_extent || other_padding_stride == dynamic_extent)`.

```
template<size_t other_padding_stride, class OtherExtents>
  constexpr explicit(not is_convertible_v<OtherExtents, extents_type>)
```

```
    mapping(const layout_left_padded<other_padding_stride>::mapping<OtherEx
```

Constraints:

- `extents_type::rank()` equals zero or one, and
- `is_constructible_v<extents_type, OtherExtents>` is `true`.

Preconditions: `other.required_span_size()` is representable as a value of type `index_type` (*[basic.fundamental]*).

Effects:

- Direct-non-list-initializes `inner-mapping_` with `other.extents()`; and
- if `extents_type::rank()` is zero, value-initializes `unpadded-extent_` ; else, initializes `unpadded-extent_` with `other.extents().extent(0)`.

[Note: Neither mapping uses the padding stride in the rank-0 or rank-1 case, so the padding stride does not affect either the constraints or the preconditions. — *end note]*

```
constexpr extents_type extents() const noexcept;
```

Effects:

- If `extents_type::rank()` is zero, equivalent to `return extents_type{};`.
- Otherwise, equivalent to `return extents_type( inner-mapping_ .extent(P_right)..., unpadded-extent_ .extent(extents_type::rank() - 1));`.

```
constexpr std::array<index_type, extents_type::rank()>
  strides() const noexcept;
```

Effects: Equivalent to `return inner-mapping_ .strides();`.

```
constexpr index_type required_span_size() const noexcept;
```

Effects: Equivalent to `return inner-mapping_ .required_span_size();`.

```
template<class... Indices>
constexpr size_t operator()(Indices... idxs) const noexcept;
```

Constraints:

- `sizeof...(Indices) == Extents::rank()` is `true`,
- `(std::is_convertible_v<Indices, index_type> && ...)` is `true`, and
- `(std::is_nothrow_constructible<index_type, Indices> && ...)` is `true`.

Precondition: `extents_type::index-cast(i)` is a multidimensional index in `extents()` ([mdspan.overview]).

Effects: Let `P` be a parameter pack such that `is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>` is `true`. Equivalent to: `return ((static_cast<index_type>(i) * stride(P)) + ... + 0);`.

[Note: Effects are also equivalent to

`return inner-mapping_ (idxs...);`, but only after the Precondition has been applied. — *end note]*

```
static constexpr bool is_always_exhaustive() noexcept;
```

Returns:

- If `extents_type::rank()` equals zero or one, `true`;
- else, if neither `<it>inner-mapping-type</it>::static_extent(extents_type::rank() - 1)` nor `extents_type::static_extent(extents_type::rank() - 1)` equal `dynamic_extent`, then `<it>inner-mapping-type</it>::static_extent(extents_type::rank() - 1) == extents_type::static_extent(extents_type::rank() - 1)`;
- otherwise, `false`.

```
constexpr bool is_exhaustive() const noexcept;
```

Returns:

- If `extents_type::rank()` equals zero, then `true`;
- else, `inner-mapping_ .extent(extents_type::rank() - 1) == unpadded-extent_ .extent(extents_type::rank() - 1)`.

```
constexpr index_type stride(rank_type r) const noexcept;
```

Effects: Equivalent to `return inner-mapping_ .stride(r);`.

§ 4.3. Layout specializations of `submdspan_mapping` [mdspan.submdspan.mapping]

> At the top of Section ﷺ [mdspan.submdspan.mapping] ("Layout specializations of `submdspan_mapping`"), before paragraph 1, add the following to the end of the synopsis of specializations.

```
template<class Extents, std::size_t padding_stride, class... SliceSpecifier
  constexpr auto submdspan_mapping(
    const layout_left_padded<padding_stride>::template mapping<Extents>& sr
    SliceSpecifiers ... slices) -> see below;

template<class Extents, std::size_t padding_stride, class... SliceSpecifier
  constexpr auto submdspan_mapping(
    const layout_right_padded<padding_stride>::template mapping<Extents>& s
    SliceSpecifiers ... slices) -> see below;
```

> In paragraph 7 (the "Returns" clause) of Section ﷺ [mdspan.submdspan.mapping] ("Layout specializations of submdspan_mapping"), replace (7.3) (the `layout_stride` fall-back return type) with the following.

(7.3) Else, if

- `decltype(src)::layout_type` is `layout_left`;
- `Extents::rank()` is greater than one;
- all the S_k except for S_0 and S_1 are `full_extent_t`;
- `is_convertible_v<` S_0 `, tuple<index_type, index_type>>` is `true`; and
- `is_convertible_v<` S_1 `, tuple<index_type, index_type>>` is `true`;

then `layout_left_padded<Extents::static_extent(0)>::template mapping(sub_ext, src.extent(0))`.

(7.4) Else, if

- `decltype(src)::layout_type` is `layout_right`;
- `Extents::rank()` is greater than one;
- all the S_k except for the two rightmost S_{r-2} and S_{r-1} are `full_extent_t`;
- `is_convertible_v<` S_{r-2} `, tuple<index_type, index_type>>` is `true`; and
- `is_convertible_v<` S_{r-1} `, tuple<index_type, index_type>>` is `true`;

then `layout_right_padded<Extents::static_extent(r - 1)>::template mapping(sub_ext, src.extent(r - 1))`.

(7.5) Else, if

- `decltype(src)::layout_type` is `layout_left_padded<padding_stride>` for some `size_t padding_stride`;
- `Extents::rank()` equals zero or one; and
- if `Extents::rank()` equals one, then S_0 is `full_extent_t` or `is_convertible_v<` S_0 `, tuple<index_type, index_type>>` is `true`;

then, `layout_left_padded<padding_stride>::template mapping(sub_ext)`.

(7.6) Else, if

- `decltype(src)::layout_type` is `layout_left_padded<padding_stride>` for some `size_t padding_stride`;
- `Extents::rank()` is greater than one;
- all the S_k except for S_0 and S_1 are `full_extent_t`;
- `is_convertible_v<` S_0 `, tuple<index_type, index_type>>` is `true`; and
- `is_convertible_v<` S_1 `, tuple<index_type, index_type>>` is `true`;

then `layout_left_padded<padding_stride>::template mapping(sub_ext, src.stride(1))`;

(7.7) Else, if

- `decltype(src)::layout_type` is `layout_right_padded<padding_stride>` for some `size_t padding_stride`;
- `Extents::rank()` equals zero or one; and
- if `Extents::rank()` equals one, then S_0 is `full_extent_t` or `is_convertible_v<` S_0 `, tuple<index_type, index_type>>` is `true`;

then, `layout_right_padded<padding_stride>::template mapping(sub_ext)`.

(7.8) Else, if

- `decltype(src)::layout_type` is `layout_right_padded<padding_stride>` for some `size_t padding_stride`;
- `Extents::rank()` is greater than one;
- all the S_k except for the two rightmost S_{r-2} and S_{r-1} are `full_extent_t`;
- `is_convertible_v<` S_{r-2} `, tuple<index_type, index_type>>` is `true`; and
- `is_convertible_v<` S_{r-1} `, tuple<index_type, index_type>>` is `true`;

then,

then `layout_right_padded<padding_stride>::template mapping(sub_ext, src.stride(r - 2))`;

(7.9) Otherwise, `layout_stride::mapping(sub_ext, sub_strides)`;

§ 4.4. Layout specializations of `submdspan_offset` [mdspan.submdspan.offset]

> At the top of Section ﷺ [mdspan.submdspan.offset] ("Layout specializations of `submdspan_offset`"), before paragraph 1, add the following to the end of the synopsis of specializations. (Note that all the specializations of `submdspan_offset` share the same wording.)

```
template<class Extents, std::size_t padding_stride, class... SliceSpecifier
  constexpr size_t submdspan_offset(
    const layout_left_padded<padding_stride>::template mapping<Extents>& sr
    SliceSpecifiers ... slices);

template<class Extents, std::size_t padding_stride, class... SliceSpecifier
```

```
  constexpr size_t submdspan_offset(
    const layout_right_padded<padding_stride>::template mapping<Extents>& s
    SliceSpecifiers ... slices);
```